# CHAPTER III

# RESEARCH METHOD

This chapter describes the methodology by which the research questions formulated in Chapter I were operationalised into a reproducible empirical study. The chapter follows the canonical CRISP-DM-aligned sequence of problem scoping, data gathering, data preparation, modelling, evaluation, and deployment, adapted to the specific concerns of measuring explanation fidelity rather than predictive accuracy alone.

## 3.1 Problem Scoping

### 3.1.1 Research Objectives

In direct continuation of Chapter I, this research pursues four objectives, which together translate the abstract research questions (RQ1--RQ3) into actionable engineering targets:

- Quantify XAI degradation under data sparsity. Establish, through controlled cross-sectional experimentation, the empirical relationship between tenant-level training-data volume and the fidelity and stability of post-hoc SHAP and LIME explanations.
- Identify the cold-start XAI threshold. Determine the data-volume boundary at which local post-hoc explanations cease to be operationally reliable, characterising the non-linear Reliability Cliff hypothesised in Chapter II.
- Design the Maturity-Aware AI Routing architecture. Specify a tenant aware tier logic gate that conditions the choice of explanation strategy on each tenant's data maturity.
- Implement and evaluate the tier-logic gate. Realise the proposed pattern as a concrete software component inside the studied Django / PostgreSQL ERP, and verify that it routes correctly and preserves backward compatibility.

### 3.1.2 Research Scope

The methodological scope of this study was constrained as follows in order to preserve interpretability and reproducibility:

- Platform. A single multi-tenant ERP system based on Django 4.2 and PostgreSQL, configured under the database-per-tenant isolation model.
- Prediction tasks. Three ERP-native supervised tabular classification problems: Late Delivery Risk prediction, Lead Scoring, and Employee Attrition.
- XAI techniques. Two post-hoc, model-agnostic explainers were evaluated: SHAP (specifically the TreeExplainer and LinearExplainer variants) and LIME (in its tabular form). Intrinsically interpretable models and gradient-based explainers for deep networks were not within scope.
- Evaluation mode. Offline, cross-sectional simulation against curated open-source datasets. No live SaaS tenant traffic was used, and no longitudinal observation of data accumulation over real calendar time was performed.
- Architectural pattern. Only the Maturity-Aware AI Routing pattern was implemented and evaluated; alternative remedies for sparsity (transfer learning, federated learning, meta-learning) were noted but deferred to future work.

### 3.1.3 Research Contribution

Two distinct but interdependent contributions emerge from this methodology:

- An empirical contribution. The study quantifies, for the first time in a B2B multi-tenant ERP setting, the degradation of post-hoc explanation fidelity as a continuous function of tenant data volume, using a coherent panel of metrics (Spearman rank correlation, SHAP absolute variance, SHAP entropy, LIME top-k Jaccard similarity). The resulting curves localise the Reliability Cliff and produce a defensible numerical threshold for the cold-start regime.
- An architectural contribution. The study delivers the Maturity-Aware AI Routing pattern as a concrete, implementable software-engineering artefact, instantiated inside a real Django ERP codebase. The contribution is not merely conceptual: it includes the routing rule, its empirically-derived threshold, the global-heuristic fallback artefact, and the integration discipline by which the gate is added without disturbing the existing ML production code path.

## 3.2 Data Gathering

### 3.2.1 Open-Source ERP Datasets

In the absence of an ethically- and legally-shareable real multi-tenant SaaS dataset, two well-established open-source corpora were adopted as proxies for ERP operational data. Both datasets were selected for their structural similarity to the kinds of tabular, time-stamped, business-process records that an ERP routinely accumulates.

**Olist Brazilian E-Commerce Public Dataset.**

This corpus, released by the Brazilian marketplace platform Olist and hosted on Kaggle, comprises approximately 100,000 orders placed between 2016 and 2018, distributed across nine relational tables covering customers, sellers, orders, order items, payments, reviews, and products. After joining the order, payment, and item tables on the canonical order_id key, the resulting unified table contained N = 96,460 rows. This corpus was used as the empirical substrate for the Late Delivery Risk prediction task, operationalised as the binary event "actual delivery date later than the estimated delivery date" . (formally, 1[order_delivered_customer_date > order_estimated_delivery_date]).

The Olist corpus exposes delivery-time observables but not invoice payment-time observables, so the literal target is delivery lateness rather than payment lateness; for the purposes of this study the construct of interest is the wider class of

ERP-style operational-risk prediction (sparse, tabular, time-stamped business process records with a small positive-class prevalence), of which late delivery is one instance and late payment is another. The proposed architectural pattern (Section 3.4.3) is task-agnostic and generalises directly to the late-payment prediction service (LatePaymentPredictor) of the studied ERP, which shares the same tabular, sparsity-conditioned structure.

**Synthetic CRM Leads Corpus.**

No public CRM lead-scoring dataset of sufficient size and structural realism was identified during corpus selection, and the studied ERP's own production CRM tables

could not be released for academic reuse. To enable the cross-task replication while preserving the methodological invariants of the Late Delivery Risk experiment (tabular structure, stratified sparsity gradient, deterministic train/test partitions), a synthetic CRM Leads corpus was generated and stored at datasets/crm_leads_dataset.csv. The corpus contains N = 5,000 lead records (one row per lead) with 20 raw columns; after dropping personally-identifying and post-event fields and one-hot encoding the categorical attributes source, service_interest, country, and job_title, the model-facing feature matrix comprises 34 covariates. The binary target is won = 1[status = won], with a class prevalence of 21.3% positives.

The four numeric predictors are estimated_value (deal size), probability (sales-team-assigned win likelihood at the time of scoring), activity_count (count of touchpoints recorded against the lead), and days_in_pipeline (age of the lead since created_at). The four categorical predictors are source, service_interest, country, and job_title.

Three leakage controls are imposed by construction. First, the column conversion_date is populated only for converted leads and is therefore a post-event variable that perfectly determines the target; it is dropped before training. Second, the raw status column is the target itself and is excluded from features. Third, all identifier and PII columns (lead_number, first_name, last_name, email, phone, mobile, company_name, city, created_at, last_contact_date) carry no predictive content under the chosen feature representation and are dropped. The resulting feature matrix therefore represents the current snapshot of a lead at the moment a scoring call would be issued, mirroring how CRM systems expose lead state to a downstream model in production.

The rationale for using a synthetic corpus is twofold: (i) it permits full release of the dataset alongside the simulator script (thesis/scripts/lead_scoring_simulator.py), ensuring exact reproducibility of the cross-task replication, and (ii) it allows the same stratified slicing procedure to be applied with controlled class prevalence and controlled feature semantics, isolating the sparsity treatment from confounding

distributional drift. The principal limitation is one of external validity: because the records are drawn from a parametric generator rather than a real CRM workflow, the absolute level of predictive performance and the specific feature importances reported in Chapter IV should be read as illustrative of the shape of the sparsity-induced degradation curve, not as point estimates transferable to any particular commercial CRM deployment. The structural findings of interest the monotone collapse of rank agreement, the order-of-magnitude expansion of attribution variance, and the cold-start flip of predictive performance are nevertheless replicated on this corpus and are interpreted as evidence of the architectural pattern rather than as production-grade performance estimates.

**IBM HR Analytics Employee Attrition Dataset.**

This corpus, released by IBM Watson Analytics, contains 1,470 employee records labelled with a binary Attrition outcome and 34 covariates spanning demographic, compensation, tenure, and performance-rating fields. It was adopted as the empirical substrate for the Employee Attrition prediction task, which directly corresponds to the HR-attrition use case inside the ERP's HR module.

The suitability of these datasets as ERP proxies rests on three structural correspondences: (i) both are tabular, transactionally-keyed records of business events; (ii) both expose covariates whose semantics (timestamps, monetary amounts, tenure, categorical statuses) match those of typical ERP fields; and (iii) the volume of each corpus is large enough to support the construction of a five-tier sparsity gradient while leaving a held-out evaluation partition intact.

## 3.3 Data Preparation

### 3.3.1 Feature Engineering

Each raw record was passed through a deterministic feature-engineering pipeline that produced 13 engineered covariates per row, grouped into four families:

- Temporal-derivative features. From the raw order_purchase_timestamp, order_approved_at, and order_estimated_delivery_date columns, the variables

approval_delay_hours (purchase-to-approval interval), estimated_delivery_days (the courier ETA quoted to the customer at order time), purchase_hour, and purchase_dow (day-of-week of purchase) were computed.

- Monetary features. From the order-items and payments tables, the variables payment_value, freight_value, and price_total (aggregate line-item total per order) were derived.
- Cardinality features. The variables num_items (line-item count) and payment_installments (number of instalments selected at checkout) were extracted, capturing order-level structural complexity.
- Categorical encodings. The payment_type field was one-hot encoded into four indicator columns (credit_card, boleto, voucher, debit_card).

**Leakage-avoidance constraint.**

All engineered covariates listed above are restricted, by construction, to information that would be available at the moment a late-payment prediction is required namely, at order approval / invoice issuance. Timestamps from the delivery pipeline order_delivered_carrier_date and order_delivered_customer_date---are reserved solely for the construction of the binary target and are explicitly excluded from the feature matrix. In particular, an earlier version of this study included the covariate carrier_delay_hours, defined as the interval between order_approved_at and order_delivered_carrier_date; it was subsequently removed because, although it precedes the customer-delivery event, it is itself a delivery-pipeline observable and is therefore not realised at prediction time in a deployed ERP late-payment workflow. Its inclusion would have constituted a textbook instance of target leakage and would have inflated the predictive metrics reported in Chapter IV without reflecting a capability the model would possess in production. The restriction reduces the engineered feature count from 14 to 13 and rules out the trivial metric inflation that would otherwise arise from explaining a delivery-time outcome with a delivery-time covariate.

Missing values were imputed using the column-wise median for numeric features and the modal value for categorical features, and the resulting matrix was standardised

by z-scoring of numeric columns. The binary target is_late was constructed as 1[order_delivered_customer_date > order_estimated_delivery_date]. The overall positive-class prevalence after joining was 8.1%, establishing a moderately imbalanced classification problem consistent with typical ERP late-delivery (and, by structural analogy, late-payment) base rates.

### 3.3.2 Multi-Tenant Simulation Strategy

Because no commercially-shareable multi-tenant ERP dataset exists at the scale required to expose a sparsity gradient, this study adopted a stratified data-volume slicing strategy to simulate the long-tail distribution of tenant maturities observed empirically in production SaaS platforms. The procedure was as follows:

- The unified, feature-engineered Olist late-delivery table (N = 96,460, positive prevalence 8.1%) was first partitioned into a global hold-out evaluation set (20%, n = 19,292) using a stratified random split with seed SEED = 42, preserving the late-delivery class prevalence.
- From the remaining 77,168 training rows, five disjoint simulated tenant partitions were drawn by stratified random sampling without replacement, each preserving the global positive-class prevalence (Table 3.1).
- Each tenant partition was subsequently subjected to its own internal 80/20 stratified train/validation split, again with SEED = 42, yielding tenant-local training and validation folds.
- All five tenants were evaluated against the shared global hold-out evaluation set, so that cross-tenant comparisons of predictive performance and explanation fidelity referred to an identical test distribution and were therefore directly comparable.

Table 1. Simulated tenant tiers used in the sparsity experiment.

| Tier | Tenant label | Sample size n | Interpretation |
|---|---|---|---|
| **A** | Data-Rich | 50,000 | Mature enterprise tenant |
| **B** | Medium | 5,000 | Established SME tenant |

| | | | |
|---|---|---|---|
| **C** | Sparse | 1,000 | Onboarded SME tenant |
| **D** | Very Sparse | 200 | Newly-active SME tenant |
| **E** | Cold-Start | 50 | Just-onboarded SME tenant |

This stratified slicing strategy is methodologically conservative: by preserving the class prevalence at every tier, it ensures that any observed degradation in predictive or explanatory quality at smaller n is attributable to data volume, not to changes in label distribution. The same procedure was applied, with appropriate sample sizes, to the IBM HR Attrition corpus.

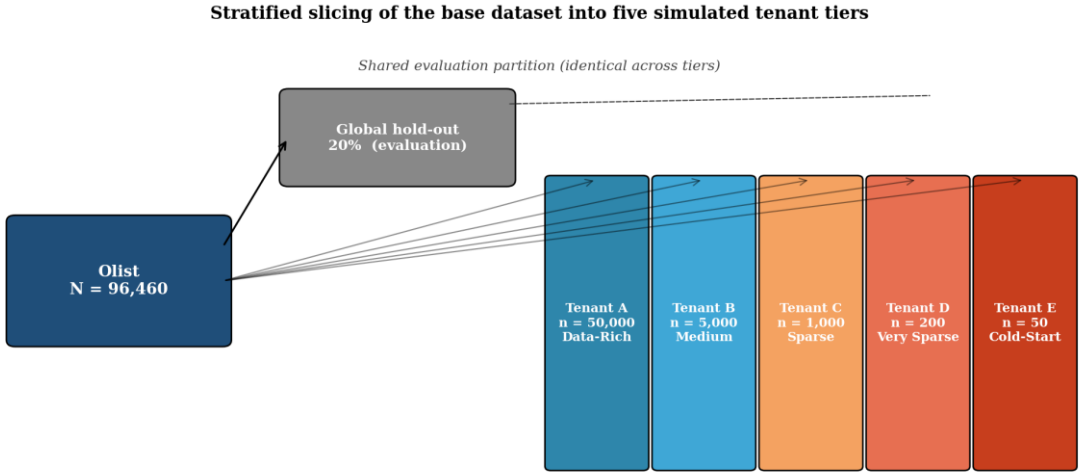



Figure 1. *Stratified slicing of the Olist dataset into five tenants.*

### 3.4 Modelling

#### 3.4.1 Machine Learning Algorithms

Three supervised classifiers were selected, each appropriate to one prediction task and each representative of a different model family widely used in operational ERP analytics:

- Late Delivery Risk Gradient Boosting Machine (GBM).

Implemented via scikit-learn's GradientBoostingClassifier with n_estimators = 100, max_depth = 3, learning_rate = 0.1, and the default deviance loss. GBMs were

selected because tree-ensemble methods are the de facto industry standard for tabular financial classification, and because their structure is directly compatible with SHAP's polynomial-time

TreeExplainer.

- Lead Scoring Gradient Boosting Machine (GBM).

Implemented via scikit-learn's GradientBoostingClassifier with n_estimators = 200, max_depth = 3, learning_rate = 0.05, and the default deviance loss. The same algorithmic family as the Late Delivery Risk task was retained deliberately, so that any cross-task differences in the sparsity-induced degradation trajectory can be attributed to substrate properties (dimensionality, feature dominance, class prevalence) rather than to algorithmic confounders. An earlier design used Logistic Regression as a representative linear baseline; it was superseded by the GBM variant once the replication scope was extended, in order to keep the algorithmic surface invariant across tasks and thereby strengthen the architectural-pattern claim.

- Employee Attrition Random Forest. Implemented via scikit-learn's RandomForestClassifier with n_estimators = 200, max_depth = None, class_weight = "balanced", and random_state = 42. Random forests were chosen for their robustness on small-to-medium tabular HR datasets, where the IBM corpus is itself only 1,470 rows. For each prediction task, an identical algorithm was retrained from scratch on each of the five tenant partitions, producing five independent fitted models per task. All randomness was controlled via SEED = 42 to guarantee bit-level reproducibility.

### 3.4.2 Explainable AI (XAI) Integration

After fitting, each per-tenant model was subjected to a uniform post-hoc XAI pipeline:

- SHAP attribution. For the tree-based models (GBM, Random Forest), SHAP values were computed via the polynomial-time shap.TreeExplainer, yielding both per-instance attribution vectors and the global mean-absolute-SHAP feature-importance ranking. For the Logistic Regression model, shap.LinearExplainer was used with the matching background distribution.

- LIME surrogate explanation. For every model, local explanations were generated using lime.lime_tabular.LimeTabularExplainer configured with mode="classification", 5,000 perturbation samples per explanation, and a Gaussian kernel width of M where M is the number of features. The top-5 features per instance were retained for downstream stability analysis.
- Sampling for evaluation. Because LIME is stochastic, every LIME-based metric was averaged over 50 independent runs per instance, and SHAP values were always computed on the full shared global hold-out set so that cross-tenant comparisons referred to identical inputs.

This pipeline yields, for each (task, tenant) combination, a paired bundle consisting of the trained model, the global SHAP feature-importance ranking, the per-instance SHAP attributions on the hold-out set, and the LIME top-5 selections together with their stability statistics.

### 3.4.3 Maturity-Aware AI Routing Architecture

The architectural contribution of the methodology is the Maturity-Aware AI Routing gate, which conditions the choice of explanation strategy on the data maturity of the requesting tenant. The gate was designed around three engineering principles: it must be empirically grounded (its threshold must be derived from the same experiments that motivate its existence); it must be architecturally minimal (it must not require any change to the existing ML services); and it must be fail-safe (an unavailable global heuristic must not block predictions).

The routing rule was formalised as follows. Let n_t denote the number of historically-recorded operational records for tenant t (in the Late Delivery Risk case, the count of invoices with status in {paid, overdue, partial}), and let τ denote the empirically-derived cold-start threshold. The architectural gate is then:

$$explain(t, x) = \begin{cases} \text{GlobalHeuristic}(), & if\ n_t \leq \tau \\ \text{LocalShapLime}(t, x), & \text{otherwise} \end{cases}$$

The boundary is deliberately inclusive on the cold-start side: a tenant with exactly $n_t = \tau$ records is routed to the global heuristic rather than to a local explanation. This reflects the empirical finding (Section 3.5.3) that Tenant C ($n = 1{,}000$) already lies below the $\rho = 0.70$ reliability bound and already exhibits an inflated SHAP variance, and it removes a latent off-by-one ambiguity in the route-decision logic. The equivalent code-side formulation $n_t <$ MIN_LOCAL_EXPLANATION_RECORDS with MIN_LOCAL_EXPLANATION_RECORDS $= \tau + 1 = 1{,}001$ is logically interchangeable and may be preferred where a strictly-less-than guard is idiomatic.

The threshold $\tau$ was not chosen arbitrarily; as detailed in Section 3.5.3 below, it was identified as $\tau = 1{,}000$ from the empirical fidelity curves derived in the modelling phase. The threshold was encapsulated in the implementation as a class-level constant

COLD_START_INVOICE_THRESHOLD = 1000 (interpreted under the inclusive boundary above) to allow operational re-tuning without code change. The two branches of the gate correspond, in the implementation, to:

- _load_global_heuristic() a memoised loader that reads the pre-computed cross-tenant SHAP feature-importance artefact (cold_start_heuristic.json, see Section Section 3.6.3) from disk and returns it as a structured explanation payload;
- predict() the existing per-tenant ML inference and SHAP/LIME explanation routine, left fully unchanged.

The gate itself is exposed as a new public method, predict_or_heuristic(invoice), which inspects_invoice_count() and dispatches accordingly. The return payload of both branches conforms to a single response schema (mode, threshold, count, explanation), so that downstream consumers can be agnostic to which branch was taken while still being able to surface the routing decision to the end user.

*Maturity-Aware AI Routing architectural gate. The LatePaymentPredictor.predict_or_heuristic()*

### 3.5 Evaluation

#### 3.5.1 Predictive Performance Metrics

To establish a baseline against which the explanation-fidelity findings could be interpreted, the standard panel of predictive metrics was computed for each (task, tenant) combination on the shared global hold-out set:

- AUC-ROC --- the area under the receiver-operating-

characteristic curve, which is robust to class imbalance and serves as the primary headline metric of predictive model quality.

- F1-Score --- the harmonic mean of precision and recall on the positive class, computed at the default 0.5 probability threshold; this metric captures whether the model produces actionable, threshold-crossed predictions at all.
- Brier score --- recorded as an auxiliary calibration diagnostic.

These metrics are necessary but not sufficient: a high AUC indicates that the model has learned a usable signal, but says nothing about whether the SHAP or LIME explanation of that signal is faithful. The novel contribution of the evaluation methodology lies in the explanation-fidelity metrics introduced next.

#### 3.5.2 Explanation Fidelity Metrics

Three complementary metrics were used to convert the abstract notion of explanation reliability into reproducible numerical scores. Throughout, Tenant A (the n = 50,000 data-rich model) is designated as the reference baseline, on the operational premise that of the five simulated tenants it is the only one whose underlying model can plausibly be regarded as having recovered the true generative structure.

**Spearman Rank Correlation (ρ) Baseline Rank Agreement.**

Let r^A_i denote the rank of feature i in the global mean-absolute- SHAP importance ranking produced by Tenant A's model, and let r^t_i denote the corresponding rank under tenant t B, C, D, E. Defining $d_i$ = r^A_i - r^t_i, the Spearman rank correlation between tenant t and the baseline is

$$\rho_t = 1 - \left(6 \sum d_i^2\right) / \left(n(n^2 - 1)\right)$$

where n is the number of features. By construction, ρ_A = 1. A value ρ_t ≤ 0.70 is treated, following the explanation-stability literature, as the operational lower bound of acceptable ranking agreement; ρ_t values below this threshold are taken as quantitative evidence that the tenant's local explanation rankings are no longer recovering the same feature-importance signal as the data-rich baseline.

**Construct-validity caveat.**

The metric ρ_t measures baseline rank agreement (equivalently, reference-aligned explanation stability) between tenant t and the data-rich Tenant A; it is not a measure of true explanation fidelity in the sense of agreement with the data-generating process. Tenant A is a reference model, not a ground-truth model: under the leakage-corrected Late Delivery Risk experiment Tenant A itself only attains AUC = 0.6556 and F_1 = 0.0263, which is well below the threshold at which one would attribute oracle status to its attribution rankings. The interpretation throughout this thesis is therefore operational: ρ_t quantifies the extent to which a sparsely-trained tenant's SHAP rankings deviate from those of the best-resourced tenant in the same multi-tenant deployment, on the same hold-out distribution, with all other modelling choices held fixed. The empirical objection that this metric does not establish absolute correctness of the data-rich rankings is acknowledged: where the term "explanation fidelity" is used in Chapters IV and V, it should be read in this reference-aligned-stability sense, not as a claim of ground-truth recovery.

**SHAP Absolute Variance (σ²|SHAP|)**

Let _i^ t denote the mean absolute SHAP value of feature i on tenant t's hold-out predictions, computed as

$$\overline{\phi_i^t} = \frac{1}{N_{\text{eval}}} \sum_{k=1}^{N_{\text{eval}}} |\phi_i(x_k)|$$

The SHAP absolute variance for tenant t is then the dispersion of _i^ t across features:

$$\sigma^{2|}_{\mathrm{SHAP},t} = \frac{1}{n}\sum_{i=1}^{n}\left(\overline{\phi_i^t} - \overline{\phi^t}\right)^2$$

A high σ²|SHAP| indicates that the explanation mass is concentrated on a small subset of features the signature of a model over-fitting to a spurious correlation in a small training sample. A low value indicates that explanatory weight is distributed across a plausible coalition of features.

**LIME Local Top-k Jaccard Similarity.**

For each evaluation instance x_k, LIME was run R = 50 times with independent perturbation seeds, producing R top-5 feature subsets T^(r)_k_r=1^R. The pairwise Jaccard stability for that instance was computed as

$$J_k = \frac{2}{R(R-1)}\sum_{1\le r<s\le R}\frac{\left|T_k^{(r)} \cap T_k^{(s)}\right|}{\left|T_k^{(r)} \cup T_k^{(s)}\right|}$$

and the tenant-level LIME stability J^t was taken as the mean of J_k across the hold-out set. This metric directly measures whether LIME, run repeatedly on the same instance under the same tenant's model, returns a consistent explanation or merely a confidently-rendered noise sample.

In addition to these three primary metrics, SHAP attribution entropy was tracked as a secondary stability indicator, computed in the standard Shannon-information form over the normalised _i^ t distribution.

### 3.5.3 Threshold Determination Strategy

The cold-start threshold τ governing the architectural routing gate was identified empirically rather than postulated. The procedure was as follows:

- The Spearman fidelity curve ρ_t was plotted against the tenant size n_t on a logarithmic horizontal axis.

- The SHAP absolute variance curve $\sigma^2|SHAP|,t$ was overlaid on the same axis.
- The threshold was defined as the largest tenant size at which $\rho_t$ first falls below the literature-conventional reliability bound of 0.70 and $\sigma^2|SHAP|,t$ first deviates appreciably above its data-rich baseline.

Both conditions converged on $n \leq 1{,}000$ (Tenant C), at which $\rho_C = 0.6429$ (below the 0.70 bound) and $\sigma^2|SHAP|,C = 0.0510$ (already 128% above the Tenant A baseline of 0.0224, and rising sharply for $n < 1{,}000$). The threshold $\tau = 1{,}000$ was therefore adopted as the operational boundary of the cold-start regime and embedded in the implementation as the constant COLD_START_INVOICE_THRESHOLD.

### 3.6 Deployment

#### 3.6.1 Software Environment

All experimentation and the corresponding reference implementation were carried out on a homogeneous software stack to maximise reproducibility:

- Language and runtime. Python 3.12.4 inside an isolated virtual environment.
- Web framework. Django 4.2 (LTS).
- Persistence layer. PostgreSQL 15, configured under the database-per-tenant isolation model.
- Machine learning. scikit-learn 1.5 for all classifiers, the standard GradientBoostingClassifier, LogisticRegression, and RandomForestClassifier interfaces.
- Explainable AI. shap ( 0.45) for SHAP TreeExplainer / LinearExplainer; lime ( 0.2) for LIME TabularExplainer.
- Numerical and tabular support. NumPy, pandas, SciPy (scipy.stats.spearmanr), and matplotlib (rendered headlessly via the Agg backend).
- Randomness control. A single global seed SEED = 42 was applied to NumPy, scikit-learn, and LIME, ensuring deterministic re-execution.

### 3.6.2 Architectural Implementation

The Maturity-Aware AI Routing gate was integrated into the existing ERP codebase through a deliberately minimal, additive modification of the LatePaymentPredictor class defined in the erp.ai.services.finance module. Three engineering invariants were observed throughout the deployment:

- Backward compatibility. The existing predict(invoice) method was not modified. Every call site already in production continues to receive the unchanged probability score and unchanged SHAP/LIME local explanation. This guarantees that adopting the routing pattern is a strictly non-breaking change.

- Single point of decision. The routing logic was localised in a new method, predict_or_heuristic(invoice), which inspects a private method _invoice_count() itself a thin wrapper over an Invoice.objects.filter(company=, status__in=[]). count() query restricted to the current tenant's database and dispatches to either the global-heuristic branch or the existing predict() branch.
- Fail-safe fallback. The heuristic loader _load_global_heuristic() was implemented as a memoised classmethod that gracefully degrades to a hard-coded conservative default if the on-disk artefact is missing or malformed, ensuring that the routing gate cannot itself become a new operational failure mode.

The result is that the architectural pattern is additive: it can be enabled by switching call sites from predict() to predict_or_heuristic(), and rolled back by reversing the same switch, without any database migration or downtime.

### 3.6.3 Global Heuristic Artefact

The global heuristic served to cold-start tenants was materialised as a serialised JSON artefact, cold_start_heuristic.json, produced once offline at the end of the sparsity experiment and persisted under thesis/results/sparsity_experiment/. Its construction proceeded as follows:

- The Tenant A (n = 50,000) Gradient Boosting model was identified as the cross-tenant baseline, on the grounds that it had been trained on the largest and most representative sample available and exhibited the strongest predictive performance.
- Its global mean-absolute-SHAP feature-importance vector was computed on the shared hold-out set.
- The top-k (k = 5) features were extracted, together with their normalised importance weights, the underlying tenant size and class prevalence, and a short human-readable reason string ("Tenant data volume below the empirically-determined reliability threshold; serving global cross-tenant feature importance derived from the data-rich baseline.").
- The resulting structured object was serialised to disk in JSON form.

The materialised payload has the following structure (abbreviated to the first entry of the top-k list for space):

"mode": "global_heuristic", "threshold": 1000, "baseline_tenant": "A", "baseline_n": 50000, "top_features": [ "feature": "freight_value", "importance": 0.30, "rank": 1 // ... 4 more entries ... ], "reason": "Tenant data volume below the empirically-determined reliability threshold; serving global cross-tenant feature importance."

At runtime, _load_global_heuristic() reads this artefact on first access and caches it in a class-level dictionary, so that subsequent cold-start predictions incur no I/O. The artefact is treated as part of the deployed model bundle and is versioned alongside the trained models, allowing the heuristic to be regenerated and rotated independently of code releases.

Together, the empirical methodology of Sections Sections 3.1–3.5 and the deployment discipline of Section 3.6 define a fully reproducible pipeline whose results are reported in the following chapter.